

```
# 12 - Slow things down a bit
clock.tick(FRAMES_PER_SECOND) # make pygame wait
```

---

### *Listing 13-1: A timer built into the main loop*

The important variables to notice in this program are:

`TIMER_LENGTH` A constant that says how long we want our timer to run

`timerRunning` A Boolean that tells us whether the timer is running

`timeStarted` The time at which the user pressed the Start button

When the user clicks Start, `timerRunning` is set to `True` ❶. We initialize the variable `startTime` to the current time. We then disable the Start button and show the message below the buttons.

Each time through the loop, if the timer is running ❷, we subtract the starting time from the current time to see how much time has elapsed since the timer started. When the amount of time elapsed becomes greater than or equal to `TIMER_LENGTH`, whatever action we want to happen when the time is up can happen. In this sample program, we enable the Start button, remove the bottom message, print a short text output, and reset the `timerRunning` variable to `False` ❸.

The code in [Listing 13-1](#) works fine . . . for a single timer. However, this is a book on object-oriented programming, so we want this to be scalable. To generalize the functionality, we'll turn the timing code into a class. We'll take the important variables, turn them into instance variables, and split the code into methods. That way, we can define and use any number of timers in a program. The `Timer` class, along with other classes used to display timing in pygame programs, are available in a module named `pyghelpers`.

## Installing pyghelpers

To install `pyghelpers`, open the command line and enter the following two commands: